

PYTHON

DATA TYPES & VARIABLES

A Beginner-Friendly Reference Guide
int | float | bool | type() | print() | strings

Prepared for Jeevi's Data Science Roadmap — Python Fundamentals
Compatible with Python 3.14 (current version)

1 Variables in Python

Naming and storing data

A **variable** is a name that refers to a value stored in memory. Python is **dynamically typed** — you don't declare a type; Python figures it out automatically based on the value you assign.

```
name = "Jeevi"      # string
age = 21             # integer
height = 5.6         # float
is_student = True    # boolean
```

Variable naming rules:

- Must start with a letter or underscore (not a digit): `score1` is valid, `1score` is not.
- Can contain letters, digits, and underscores only — no spaces or symbols.
- Case-sensitive: `Age` and `age` are different variables.
- Avoid Python keywords like `class`, `for`, `if`.
- Convention: use `snake_case` for variable names, e.g. `student_name`.

Multiple assignment:

```
x, y, z = 1, 2, 3
a = b = c = 0      # all three set to 0
```

★ Tip

Use `type(variable_name)` any time you're unsure what kind of data a variable is holding — it's the fastest way to debug type errors.

2 Core Data Types

int, float, boolean, type(), print()

2.1 Integers (int)

Whole numbers, positive or negative, with no decimal point and no size limit (Python automatically handles very large integers).

```
marks = 95
temperature = -4
big_number = 123_456_789    # underscores improve readability
print(type(marks))          #
```

2.2 Floating-Point Numbers (float)

Numbers with a decimal point, used for measurements, percentages, prices, and any value requiring precision beyond whole numbers.

```
gpa = 8.75
price = 499.0
scientific = 2.5e3           # 2500.0 (scientific notation)
print(type(gpa))             #

# int <-> float conversion
int(9.8)      # 9    (decimal part is dropped, not rounded)
float(9)      # 9.0
```

2.3 Booleans (bool)

Represents one of exactly two values: True or False. Booleans are the result of comparisons and are commonly used in conditions. Internally, bool is a subtype of int — True == 1 and False == 0.

```
is_passed = True
has_scholarship = False
print(5 > 3)      # True
print(10 == 20)   # False
print(type(is_passed))#
print(True + True)    # 2
```

2.4 type() Function

Returns the data type of any value or variable. Extremely useful while debugging.

```
print(type(10))      #
print(type(3.14))    #
print(type("hi"))    #
print(type(True))    #
```

2.5 print() Function

Displays output to the console. Accepts multiple values, a custom separator (sep), and a custom line ending (end).

```
print("Hello", "World")          # Hello World
print("a", "b", "c", sep="-")    # a-b-c
print("Loading", end="...")      # no newline after
print("Score:", 95, "/", 100)    # mixes types, auto-converted to text
```

3 Strings

Text data, indexing & slicing

A **string** (`str`) is a sequence of characters enclosed in single, double, or triple quotes. Strings are **immutable** — once created, their contents cannot be changed in place; string methods always return a *new* string.

```
name = 'Jeevi'
greeting = "Hello, Data Scientist!"
paragraph = """This is a
multi-line string"""
```

3.1 Indexing

Each character in a string has a position (index), starting at 0 from the left. Negative indices count from the right, starting at -1.

```
text = "Python"
text[0]    # 'P'    (first character)
text[5]    # 'n'    (last character)
text[-1]   # 'n'    (last character, from the right)
text[-6]   # 'P'    (first character, from the right)
```

3.2 Slicing

Slicing extracts a substring using `string[start:stop:step]`. `start` is inclusive, `stop` is exclusive. Any part can be omitted to use its default (start of string, end of string, or step 1).

```
text = "Data Science"
text[0:4]    # 'Data'    characters 0,1,2,3
text[5:]     # 'Science' from index 5 to the end
text[:4]     # 'Data'    from the start to index 3
text[-7:]    # 'Science' last 7 characters
text[::-2]   # 'Dt cec'  every 2nd character
text[::-1]   # 'ecneicS ataD' reversed string
```

★ Tip

Slicing never raises an error for out-of-range indices, so `text[0:100]` safely returns the whole string even if it's shorter than 100 characters.

4 Essential String Methods

len, upper, lower, strip, split, replace

String methods are called using dot notation: `string.method()`. Since strings are immutable, every method below returns a brand-new string or value rather than modifying the original.

Method	What it does	Example -> Result
<code>len()</code>	Returns the number of characters in a string (built-in function, not a method).	<code>len("Python") -> 6</code>
<code>.upper()</code>	Converts all characters to uppercase.	<code>"data".upper() -> "DATA"</code>
<code>.lower()</code>	Converts all characters to lowercase.	<code>"DATA".lower() -> "data"</code>
<code>.strip()</code>	Removes leading/trailing whitespace (or given characters).	<code>" hi ".strip() -> "hi"</code>
<code>.split()</code>	Splits a string into a list, using whitespace or a given separator.	<code>"a,b,c".split(",") -> ['a', 'b', 'c']</code>
<code>.replace()</code>	Replaces all occurrences of a substring with another.	<code>"cat".replace("c","b") -> "bat"</code>

Worked examples:

```
s = "  Learning Python is Fun  "

len(s)                # 30 (includes the spaces)
s.strip()              # 'Learning Python is Fun'
s.strip().upper()      # 'LEARNING PYTHON IS FUN'
s.strip().lower()      # 'learning python is fun'
s.strip().split()      # ['Learning', 'Python', 'is', 'Fun']
s.strip().replace("Fun", "Powerful")
                        # 'Learning Python is Powerful'
```

Extra useful methods worth knowing: `.startswith()`, `.endswith()`, `.find()`, `.count()`, `.join()`, and `.isdigit()`.

5 f-strings & String Formatting

Building dynamic text output

5.1 f-strings (recommended, Python 3.6+)

An **f-string** lets you embed expressions directly inside a string by prefixing it with `f` and wrapping the expression in curly braces `{ }`. This is the modern, most readable way to format strings.

```
name = "Jeevi"
score = 92.567

print(f"Hello, {name}!")
# Hello, Jeevi!

print(f"Score: {score}")
# Score: 92.567

print(f"{name} scored {score:.2f}%")    # 2 decimal places
# Jeevi scored 92.57%

print(f"Next year age: {21 + 1}")        # expressions work too
# Next year age: 22
```

5.2 Common Format Specifiers

Specifier	Meaning	Example -> Result
<code>:.2f</code>	Fixed 2 decimal places	<code>f"{3.14159:.2f}"</code> -> <code>"3.14"</code>
<code>:,</code>	Thousands separator	<code>f"{1000000:,}"</code> -> <code>"1,000,000"</code>
<code>:>10</code>	Right-align in 10 chars	<code>f"{'hi':>10}"</code> -> <code>" hi"</code>
<code>:<10</code>	Left-align in 10 chars	<code>f"{'hi':<10}"</code> -> <code>"hi "</code>
<code>^10</code>	Center-align in 10 chars	<code>f"{'hi':^10}"</code> -> <code>" hi "</code>
<code>:d</code>	Format as integer	<code>f"{7.0:.0f}"</code> -> <code>"7"</code>
<code>%</code>	Percentage format	<code>f"{0.856:.1%}"</code> -> <code>"85.6%"</code>

5.3 Older Formatting Styles (good to recognise)

```
# .format() method
"Hello, {}".format(name)          # 'Hello, Jeevi!'
"{0} is {1} years old".format(name, 21) # 'Jeevi is 21 years old'

# % operator (legacy, still seen in older code)
"Hello, %s!" % name               # 'Hello, Jeevi!'
"Score: %.2f" % score             # 'Score: 92.57'
```

★ Best Practice

Prefer f-strings for all new code — they are the fastest, most readable, and the current standard in Python 3.14.

6 Quick Recap Cheat-Sheet

One page to revise it all

Data Types

Type	Example	type() output
int	age = 21	
float	gpa = 8.75	
bool	is_ok = True	
str	name = 'Jeevi'	

String Toolkit

```
len(s)           # length
s.upper()        # UPPERCASE
s.lower()        # lowercase
s.strip()        # remove outer whitespace
s.split(",")     # string -> list
s.replace("a","b") # substitute text
s[start:stop:step] # slicing
f"{value:.2f}"   # formatted output
```

★ Practice Task

Take your name and age as variables, then use an f-string to print: 'Hi, I'm <name>, and in 5 years I'll be <age+5>.'
Then try uppercasing your name and slicing out its first 3 letters.